

BOARD OF INTERMEDIATE EDUCATION, KARACHI

Computer Science Class XI Lectures

Units 1 to 7 | Sindh Curriculum 2019

Seven classroom lectures for Computer Science Class XI, written for the Board of Intermediate Education, Karachi.

The lectures follow the Sindh Curriculum for Computer Science, Grades XI-XII, 2019: the computer system, memory, the system unit, operating systems, C++ programming, arrays and structures, and networks. They are original teaching notes. They are not a reproduction of a textbook.

Every complete C++ program in these notes was compiled and run. Sample inputs and the output to expect are stated beside the programs.

The board paper associated with this course is three hours and 75 marks: 29 multiple-choice questions, ten short answers chosen from fifteen, and two detailed answers chosen from three. Before the annual examination, confirm the model paper and any reduced syllabus for your year at biek.edu.pk.

Units inside this file: 1 Overview of the computer system. 2 Computer memory. 3 Inside the system unit. 4 Operating systems. 5 Programming in C++. 6 Arrays, strings and structures. 7 Computer communication and networks.

Unit	Lecture	Weight
1	Overview of the computer system	15%
2	Computer memory	10%
3	Inside the system unit	15%
4	Operating systems	10%
5	Programming in C++	20%
6	Arrays, strings and structures	15%
7	Computer communication and networks	15%

Class XI | before the units

How to use these lectures

Paper pattern used with the published BIEK scheme for Computer Science

These lectures follow the Sindh Curriculum for Computer Science, Grades XI-XII, 2019, which is the curriculum taught for the Board of Intermediate Education, Karachi. Each unit states the outcomes, teaches the ideas in classroom language, and ends with the kind of questions the board paper asks. They are original teaching notes, not a copy of any textbook.

What you should be able to do

- Know the shape of the Computer Science paper and the weight of each Class XI unit.
- Write a definition, a difference, and a program in the form examiners can mark.

Path through the lecture

- 1 Read the lecture once without stopping on the questions.
- 2 Close the notes and answer the checkpoint. Then mark it with the key.
- 3 Type every complete program and run it. The practical journal has the lab steps.
- 4 Before the exam, confirm the model paper and any reduced syllabus for your year on the board site biek.edu.pk.

The paper

The scheme published by the Board for Computer Science Paper I and Computer Science Paper II sets a three-hour paper of 75 marks. Use that shape for practice. If the board issues a new model paper for your examination year, follow the new paper.

Section	Time guide	What you do	Marks
A	30 minutes	29 multiple-choice questions. Attempt all of them.	29
B	90 minutes	15 short questions are set. Attempt any 10. Each is 3 marks.	30
C	60 minutes	3 detailed questions are set. Attempt any 2. Each is 8 marks. The scheme does not split these into parts.	16

Section A is about 40 percent of the paper, Section B about 40 percent, and Section C about 20 percent. A paper of this shape rewards a student who can define a term, give an example, and still write a working program.

Unit weight in Class XI

The curriculum gives these weights. A higher weight means more classroom time and a greater chance of a longer question.

Unit	Title	Weight	Theory periods	Lab periods
1	Overview of the computer system	15%	15	-
2	Computer memory	10%	10	-
3	Inside the system unit	15%	10	10
4	Operating system	10%	10	10

Unit	Title	Weight	Theory periods	Lab periods
5	Programming in C++	20%	20	20
6	Arrays, strings and structures	15%	25	25
7	Communication and networks	15%	25	-

How answers earn marks

- **Define** means one precise sentence, then stop. Add an example only if the question asks for it.
- **Differentiate** means a table. Three rows are enough for 3 marks. Name the basis of comparison in the first column.
- **Explain** or **describe** means a short paragraph with a reason, not a list of headings only.
- **Write a program** means a complete program: include line, the code, and a return from main. Use the variable names the question gives.
- Draw a diagram when the question says show, draw, or illustrate. Label every box.

Exam point

Many colleges still use an old compiler that wants `#include <iostream.h>` and `void main()`. These lectures use standard C++, which current compilers accept: `#include <iostream>`, using `namespace std;`, and `int main()`. The logic the examiner marks is the same. Match the house style of your practical journal, and keep the program correct.

Bring it together

- Practice all three sections. Programming units carry the most weight in Class XI.
- Check the current model paper on biek.edu.pk before the annual examination.

Class XI | Unit 1 | weight 15%

Overview of the computer system

Board weight about 15% | 15 theory periods

A computer is a system, not a single box. This lecture names the parts, sorts the devices by the job they do, and separates the programs that run the machine from the programs that do your work.

What you should be able to do

- Describe a computer and the four basic operations.
- Draw the block diagram and place hardware, software and memory on it.
- Classify input, output, storage, processing and communication devices.
- Explain system software, application software and utility software.
- Compare primary and secondary memory.
- Describe how computing is used now, including artificial intelligence and cloud computing.

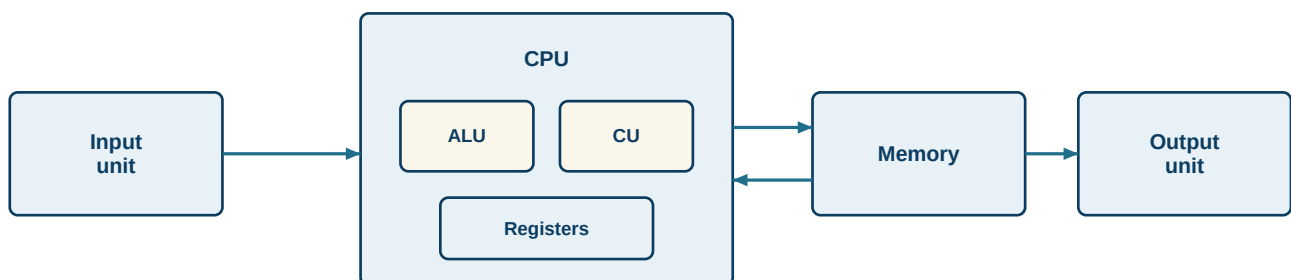
Path through the lecture

- 1 Start with the definition and the block diagram. Students copy the diagram before any list of devices.
- 2 Sort devices by function. A device can be named only after its class is clear.
- 3 Teach the three software families with one example each.
- 4 Close with artificial intelligence and the cloud, each tied to a local example.

1.1 A computer system

Computer. An electronic system that takes data as input, processes it according to instructions, stores data and results, and gives information as output.

Data is the raw fact you enter, such as a set of marks. Information is the processed result, such as a grade or a class average. The machine follows a program, which is a stored sequence of instructions. The four operations you should be able to name in order are input, processing, storage and output.



Data moves in. The CPU processes it with memory. Results go out.

Figure 1.1 Block diagram of a computer system.

The arrows in Figure 1.1 are the basic operations. Input devices send data in. The CPU processes it. Memory holds the instructions and the data while the work is going on. Output devices present the result. Secondary storage keeps a copy after the power is switched off. Software is the set of programs that tell this hardware what to do. Hardware is the physical equipment.

1.2 Hardware by function

Classify a device by the job it does. A keyboard is input because data travels from the user into the computer. A printer is output because information travels out. A hard disk is storage. The processor is the processing device. A router is a communication device.

Input devices

- **Keyboard** enters characters. A mouse points and clicks.
- **Joystick or joypad** is used for direction and actions, often in games and simulators.
- **Scanning devices** turn a printed page or a photograph into digital data.
- **Data tablet and stylus** let a person draw or write by hand. The stylus is the pen.
- **Voice recognition** turns speech into text or commands. A **microphone** captures the sound.
- **Touch screen** receives input by touch and often displays output as well. Name the role the question asks about.
- **Webcam** captures live video. A **barcode scanner** reads a printed code on a product.

Output devices

- **Monitors** display results. LED and LCD panels are flat and use less power. Plasma is an older flat panel. CRT is the deep glass monitor of earlier computers.
- **Printers** put results on paper. A dot-matrix printer is an impact printer: pins strike a ribbon. Inkjet and laser printers are non-impact. Thermal printers heat special paper.
- A **plotter** draws precise lines, used for maps and engineering drawings.
- A **multimedia projector** throws the screen onto a wall. **Speakers and headphones** produce sound.
- A **speech generating device (SGD)** speaks text aloud for a user who cannot use ordinary speech.

Storage, processing and communication

- **Hard disk** stores large amounts of data magnetically. **Flash, USB and pen drives** store data in electronic memory and travel in a pocket.
- **CD, DVD, CD-R and CD-RW** are optical discs. CD-R records once. CD-RW can be rewritten. **Blu-ray** holds high-definition video. **SD and microSD cards** are small flash cards used in phones and cameras.
- The **processor** is the processing hardware. It is taught properly in Unit 3.
- **Network interface card** connects the computer to a network cable or a wireless network.
- A **modem** turns the computer's digital signal into a form a telephone or cable line can carry, and turns the incoming signal back.
- **Data terminal equipment** is the device at the end of a link, such as a computer or a terminal. A **fax machine** sends a copy of a page over a telephone line.
- A **hub** repeats data to every connected device. A **switch** sends a frame only toward its destination inside a local network. A **router** forwards packets between networks. A **gateway** connects networks that do not speak the same protocol and translates between them.

Exam point

If a question asks you to classify webcam, plotter, SD card, router and keyboard, write the class first and the name second: input, output, storage, communication, input.

1.3 Software

Software. The programs and related data that tell the hardware what to do.

System software runs and controls the computer. It includes the operating system, device drivers, language translators and antimalware programs. A device driver teaches the operating system how to talk to one piece of hardware, such as a printer. A language translator turns a program written by a person into a form the processor can run.

Language translators

Translator	What it does
Assembler	Converts assembly language into machine code.
Compiler	Translates a whole high-level program into an object program, then the program can be run.
Interpreter	Translates and runs one statement at a time. It stops at the first error it meets.

Antimalware protects the computer from harmful programs. Antivirus software finds and removes viruses. A Trojan horse is a harmful program disguised as something useful, so protection has to recognise that disguise as well.

Application software does a job for the user. General-purpose software is sold for many people and many tasks: a word processor, a spreadsheet, a browser. Special-purpose software is built for one kind of organisation: a college admission system, a hospital record system, a shop billing system.

Utility software maintains the computer. Compression software makes files smaller for storage or sending. Data-recovery software tries to bring back deleted or damaged files. A disk manager partitions a disk, cleans up files and can defragment a hard disk.

1.4 Primary and secondary memory

Primary memory is the memory the CPU works with directly. RAM holds the running program and its data. It is fast, and it is volatile: the contents go when the power goes. ROM holds permanent start-up instructions and is non-volatile.

Secondary memory keeps data and programs for the long term. Hard disks, USB drives, optical discs and memory cards are secondary. The CPU does not execute a program from a USB drive directly. The program is first loaded into RAM. Secondary memory has more capacity, costs less per byte, and keeps its contents without power. It is slower than RAM.

Primary memory and secondary memory

Basis	Primary	Secondary
Contact with the CPU	Direct	Indirect, through primary memory
Speed	Faster	Slower
Typical contents	Running program and data	Stored files and software
Power removed	RAM loses its contents	Contents remain
Capacity and cost	Smaller, costlier per byte	Larger, cheaper per byte

1.5 Computing now

Computers now sit inside phones, vehicles, hospitals, banks and classrooms. They speed up calculation, store huge records, and connect people. The same tools raise questions of privacy, honest use of information, and dependence on a working network.

Artificial intelligence. The building of computer systems that perform tasks which ordinarily need human intelligence, such as recognising speech, learning from examples, and supporting a decision.

In a hospital, an AI system can flag an X-ray for a doctor to review. In a school it can practise language with a student. In traffic it can change signal timing. On a farm it can watch crop images for disease. The system is a tool for a person who remains responsible for the decision.

Cloud computing. The delivery of storage, software and processing over a network, from servers run by a provider, paid for and used as they are needed.

- A college can start with a small service and increase storage or processing when admissions begin.
- Students reach the same files from a lab computer or a phone.
- The provider maintains the servers and keeps backup copies as part of the service.
- Several people can work on one document.
- The college spends less on its own server room at the start.

Cloud work needs a network connection. The data lives on the provider's machines, so the college needs a clear agreement about who may see it.

Exam point

A strong 8-mark answer on this unit is the block diagram with labels, plus one paragraph that places software and both kinds of memory on the diagram.

Key terms

Term	Meaning to write in an answer
Data	Raw facts entered into the computer.
Information	Data after processing, useful for a decision.
Hardware	Physical parts of the computer system.
Software	Programs that instruct the hardware.
System software	Programs that operate and control the computer.
Application software	Programs that perform a user task.
Utility software	Programs that maintain the computer.
Volatile memory	Loses its contents when power is removed.
Cloud computing	On-demand computing services delivered over a network.

Bring it together

- A computer inputs, processes, stores and outputs. Hardware does the physical work. Software gives the instructions.
- Name every device by its function: input, output, storage, processing or communication.
- System software runs the machine. Application software does the user's job. Utilities maintain the machine.
- Primary memory serves the CPU now. Secondary memory keeps files for later.
- AI systems support intelligent tasks. Cloud services provide computing over a network.

Do this in the practical

- 1 On a real system unit, point to the processor area, the memory modules, the hard drive and one port. Say which block of Figure 1.1 each part belongs to.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. Which operation turns data into information?

- A. Input
- B. Processing
- C. Storage only
- D. Printing only

2. A plotter is classified as:

- A. Input
- B. Output
- C. Processing
- D. Communication

3. Which program translates a high-level program one statement at a time?

- A. Compiler
- B. Assembler
- C. Interpreter
- D. Device driver

4. A college fee system written for that college only is:

- A. Special-purpose application software
- B. An operating system
- C. A device driver
- D. ROM

5. RAM is described as volatile because:

- A. It is very large
- B. Its contents are lost when power is removed
- C. It stores files permanently
- D. It cannot be read

6. Which device forwards packets from one network to another?

- A. Plotter
- B. Router
- C. Stylus
- D. Compiler

7. Cloud computing provides resources:

- A. Only by replacing the keyboard
- B. Over a network, as they are needed
- C. Only on a CD-ROM
- D. Only inside ROM

8. Compression software is a:

- A. Utility
- B. Language translator
- C. Input device
- D. Protocol

Answer key

Q	Answer	Why this is the answer
1	B	Processing applies instructions to data and produces information.
2	B	A plotter presents results, usually as a precise drawing.
3	C	An interpreter translates and executes statement by statement.
4	A	It performs one organisation's task, so it is special-purpose application software.
5	B	Volatile memory needs power to hold its contents.
6	B	A router connects networks and forwards packets.
7	B	The provider delivers storage, software or processing over a network.
8	A	Compression maintains files by making them smaller. That is a utility.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Define a computer.

Model answer

A computer is an electronic system that accepts data, processes it according to stored instructions, stores data and results, and produces information as output.

Q2. Differentiate compiler and interpreter.

Model answer

A compiler translates the whole source program and produces an object program which is then executed. An interpreter translates and executes one statement at a time and reports an error as soon as it reaches it. A compiled program is generally ready to run again without translating the source each time.

Q3. Give two differences between primary and secondary memory.

Model answer

Primary memory is directly accessible by the CPU and is faster. RAM, which is primary memory, loses its contents when power is removed. Secondary memory keeps files after power is removed, has larger capacity, and is slower. Programs are loaded from secondary memory into primary memory before execution.

Q4. What is a device driver?

Model answer

A device driver is system software that lets the operating system control a particular hardware device, such as a printer or a network card.

Q5. Write two advantages of cloud computing.

Model answer

The user can increase storage or processing when the work grows, and can reach the same files from different places. The provider maintains the servers, so the user spends less on a private machine room at the start.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Draw the block diagram of a computer system and explain its parts.

- 1 Define a computer in one sentence.
- 2 Draw input, CPU, memory and output. Inside the CPU show ALU, control unit and registers. Show a double arrow between CPU and memory.
- 3 Explain input as data entering, processing as the CPU following instructions, memory as the working store, and output as information leaving.
- 4 Add secondary storage under memory and say that it keeps files when power is off.
- 5 Close with one line: hardware is the equipment in the diagram, software is the instructions that make it work.

Question 2. Explain system software, application software and utility software with examples.

- 1 Define software.
- 2 System software operates the computer: operating system, device drivers, translators (assembler, compiler, interpreter) and antivirus.
- 3 Application software does user work. General-purpose examples include a word processor and a browser. A special-purpose example is a hospital system.
- 4 Utility software maintains the machine: compression, data recovery and disk management.
- 5 Give one situation in which a student uses one program from each family on the same day.

Class XI | Unit 2 | weight 10%

Computer memory

Board weight about 10% | 10 theory periods

Memory is where the computer holds bits. This lecture teaches the units used to measure that holding, then the chips the CPU uses, then the devices that keep work after the computer is switched off.

What you should be able to do

- Use bit, nibble, byte, kilobyte, megabyte, gigabyte and terabyte correctly.
- Distinguish volatile and non-volatile memory.
- Explain SRAM, DRAM, ROM, PROM, EPROM and EEPROM.
- Describe hard drives, USB flash drives, external hard drives, memory cards and combo drives.

Path through the lecture

- 1 Practise the unit ladder with two numerical conversions before naming chips.
- 2 Teach volatile and non-volatile with the power cable as the test.
- 3 Compare SRAM with DRAM, then the ROM family, in tables students can reproduce.
- 4 Finish with secondary devices they can point to in the lab.

2.1 Units of memory

A **bit** is one binary digit, 0 or 1. It is the smallest unit of storage. A **nibble** is 4 bits. A **byte** is 8 bits and is the usual unit for one character in the older character codes.

The ladder used in this course

Unit	Equals
1 nibble	4 bits
1 byte	8 bits
1 kilobyte (KB)	1024 bytes
1 megabyte (MB)	1024 KB
1 gigabyte (GB)	1024 MB
1 terabyte (TB)	1024 GB

In this course the multiplier is 1024, which is 2 to the power 10. Worked example: a file of 2 KB contains $2 \times 1024 = 2048$ bytes, and $2048 \times 8 = 16384$ bits. Worked example the other way: 4096 MB is $4096 / 1024 = 4$ GB.

Exam point

Write the unit in the answer. A bare 2048 is not a complete conversion if the question asked for bytes.

2.2 Primary memory

Volatile memory. Memory that loses its stored contents when the power supply is removed.

Non-volatile memory. Memory that keeps its stored contents when the power supply is removed.

RAM is volatile working memory. ROM and secondary storage are non-volatile. The test is simple: switch the power off and ask whether the contents are still there.

RAM

RAM. Random access memory. The processor can reach any location directly, without reading all the earlier locations. It holds the program and data that are in use, and it is volatile.

Static RAM (SRAM) stores each bit in a small circuit of transistors arranged as a flip-flop. It does not need a refresh. It is fast and costly, so it is used in small amounts as cache inside or beside the processor.

Dynamic RAM (DRAM) stores each bit as a charge in a tiny capacitor. The charge leaks, so the memory must be refreshed many times a second. DRAM is slower than SRAM and much cheaper per bit, so the large main memory of a PC is DRAM.

SRAM and DRAM

Basis	SRAM	DRAM
Cell	Flip-flop made of transistors	Capacitor and transistor
Refresh	Not required	Required
Speed	Faster	Slower
Cost and density	Costlier, fewer bits on a chip	Cheaper, more bits on a chip
Typical use	Cache	Main memory

ROM and its family

ROM. Read-only memory. In ordinary work the processor reads it and does not rewrite it. It is non-volatile and often holds the start-up program.

- **PROM**, programmable ROM, is written once by the user with a special device. After that the contents stay fixed.
- **EPROM**, erasable programmable ROM, can be erased by ultraviolet light through a window on the chip and then programmed again.
- **EEPROM**, electrically erasable programmable ROM, is erased and rewritten by electrical signals. A location can be updated without removing the chip. Flash memory, used in pen drives and SSDs, is a form of EEPROM erased in blocks.

The ROM family

Type	Written	Erased
ROM	At manufacture	Not erasable in ordinary use
PROM	Once, by the user	Not erasable
EPROM	By a programmer	By ultraviolet light
EEPROM	Electrically	Electrically

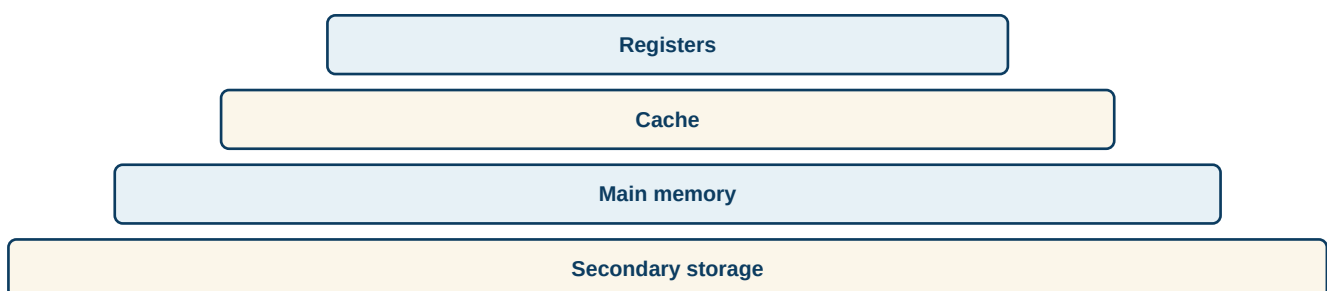


Figure 2.1 Memory ladder. The top is the fastest and the smallest. The bottom is the slowest and the largest.

2.3 Secondary memory

Secondary memory. Non-volatile storage that keeps programs and data for later use. It is larger and slower than main memory. The CPU uses its contents after they are loaded into RAM.

- A **hard disk drive** stores data on magnetic platters. A head reads and writes while the platter spins. Capacity is large. It is the traditional store inside a desktop system unit.
- A **USB flash drive**, also called a pen drive or data traveller, uses flash memory. It is portable, has no spinning part, and plugs into a USB port.
- An **external hard drive** is a hard disk in its own case, usually attached by USB, used for backup and for moving large folders.
- An **SD card** or **microSD card** is a flash card used in phones, cameras and some laptops.
- A **combo drive** is an optical drive that works with more than one disc type, commonly CD and DVD. Discs are removable. CD-R records once. CD-RW can be rewritten.

Choose the device from the job. A running program needs RAM. A file you will open next month needs secondary storage. A copy you will carry to college needs a USB drive or a memory card. A full backup of a lab machine needs an external hard drive or a large internal disk.

Exam point

Do not call a hard disk ROM. Both keep data without power. ROM is primary semiconductor memory for permanent instructions. A hard disk is secondary magnetic storage for files.

Key terms

Term	Meaning to write in an answer
Bit	A single binary digit, 0 or 1.
Nibble	Four bits.
Byte	Eight bits.
RAM	Volatile working memory with direct access to any location.
SRAM	Fast RAM that stores bits in flip-flops and needs no refresh.
DRAM	RAM that stores bits as charge and needs refresh.
ROM	Non-volatile memory read in ordinary operation.
EEPROM	ROM that can be erased and rewritten electrically.
Cache	Small, fast SRAM used to feed the processor.

Bring it together

- Measure memory from the bit up. In this course 1 KB is 1024 bytes.
- RAM is volatile working memory. SRAM is cache. DRAM is main memory.
- ROM, PROM, EPROM and EEPROM are non-volatile. They differ in how they are programmed and erased.
- Hard disks, flash drives, memory cards and optical combo drives are secondary storage.

Do this in the practical

- 1 Convert 3 KB to bytes and to bits on paper. Then read the size of a real file in the operating system and say whether it is closer to a KB, an MB or a GB.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. A nibble contains:

- A. 2 bits
- B. 4 bits
- C. 8 bits
- D. 1024 bits

2. How many bytes are there in 1 KB in this course?

- A. 1000
- B. 1024
- C. 8
- D. 512

3. Which memory needs to be refreshed?

- A. SRAM
- B. DRAM
- C. DVD
- D. PROM

4. EPROM is erased by:

- A. Ultraviolet light
- B. A magnet
- C. A paper jam
- D. A plotter

5. Which memory is typically used as cache?

- A. DRAM
- B. SRAM
- C. CD-R
- D. Blu-ray

6. A pen drive is:

- A. Volatile primary memory
- B. Secondary flash storage
- C. A language translator
- D. An input-only device

7. PROM can be programmed:

- A. Every time the PC boots
- B. Once by the user
- C. Only by ultraviolet light
- D. Only by a router

8. Which statement about ROM is correct?

- A. It is volatile
- B. It commonly holds start-up instructions
- C. It is a magnetic platter
- D. It is erased by deleting a folder

Answer key

Q	Answer	Why this is the answer
1	B	A nibble is half a byte, so 4 bits.
2	B	The course uses 1024, which is 2 to the power 10.
3	B	DRAM stores charge in capacitors, and that charge must be refreshed.
4	A	EPROM is erased by ultraviolet light shone through its window.
5	B	SRAM is fast enough and costly enough that it is used in small cache.
6	B	A pen drive keeps files without power and uses flash memory.
7	B	PROM is programmable once. After programming it stays fixed.
8	B	ROM is non-volatile and is used for permanent start-up code.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Convert 4 MB into KB.

Model answer

1 MB = 1024 KB, so 4 MB = 4 x 1024 = 4096 KB.

Q2. Differentiate SRAM and DRAM.

Model answer

SRAM stores each bit in a transistor flip-flop, does not need refresh, and is faster and costlier. It is used as cache.
DRAM stores each bit as a charge, needs refresh, and is cheaper per bit. It is used as main memory.

Q3. Differentiate EPROM and EEPROM.

Model answer

EPROM is erased by ultraviolet light and is then reprogrammed. EEPROM is erased and rewritten by electrical signals, so it can be updated in the circuit. Both are non-volatile programmable ROM.

Q4. Why is RAM called volatile?

Model answer

RAM needs electrical power to hold its contents. When power is removed, the stored program and data disappear.

Q5. What is a combo drive?

Model answer

A combo drive is an optical drive that reads or writes more than one disc type, commonly both CD and DVD.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain the units of memory from bit to terabyte, with one numerical example.

- 1 Define bit, nibble and byte with the numbers 1, 4 and 8.
- 2 Give the ladder KB, MB, GB and TB, each 1024 of the unit below.

- 3 Work one conversion in full, for example $2 \text{ KB} = 2048 \text{ bytes} = 16384 \text{ bits}$. Show the multiplication.
- 4 Say why a standard matters: file sizes and memory chips are compared with these units.

Question 2. Describe RAM and ROM, including SRAM, DRAM, PROM, EPROM and EEPROM.

- 1 Define RAM as volatile direct-access working memory, and ROM as non-volatile memory read during ordinary work.
- 2 Contrast SRAM and DRAM with cell, refresh, speed, cost and use.
- 3 Describe PROM once, EPROM with ultraviolet erase, and EEPROM with electrical erase.
- 4 Give the job of each: DRAM as main memory, SRAM as cache, ROM family as permanent or semi-permanent instructions and firmware.

Class XI | Unit 3 | weight 15%

Inside the system unit

Board weight about 15% | 10 theory periods and 10 lab periods

The system unit is the case that holds the working parts of a desktop computer. This lecture opens the case in words: the board, the processor, the buses, the ports and the slots.

What you should be able to do

- Describe CPU casings and the main parts inside the system unit.
- Name the components on the motherboard and the job of each.
- Explain the internal architecture and the external pins of a microprocessor.
- Read a processor specification.
- Define address, data and control buses and say which way data flows.
- Describe serial, parallel, USB and HDMI ports, and PCI, EISA and VESA slots.

Path through the lecture

- 1 Point at a real case before drawing the motherboard.
- 2 Teach the processor as one chip with a clear inside and a clear outside.
- 3 Use one numerical example of an address bus.
- 4 Finish in the lab by seating RAM and identifying one port.

3.1 The system unit

System unit. The case that houses the motherboard, processor, memory, storage, power supply, ports and cooling of a desktop computer.

Cases differ by size and by how easily they can be expanded. A **tower** stands on the floor, has room for drives and cards, and the fans can move air through a tall space. A **desktop** case lies flat and can hold a monitor, with less room to expand. A **small-form** case saves desk space and accepts fewer cards. An **all-in-one** builds the computer into the monitor housing, which is tidy and harder to upgrade.

- The **power supply** converts mains AC into the low-voltage DC the parts use, and it feeds them through labelled connectors.
- The **motherboard** is the main circuit board. The processor, memory and cards are mounted on it or plugged into it.
- The **hard drive** stores files. A **combo drive** reads optical discs.
- **Ports** are the sockets on the edge of the case. The **cooling system** is the heat sink and fan on the processor, plus the fans that move air through the case.

3.2 The motherboard

Every important signal of the computer meets on the motherboard. Learn the part and the job together.

Parts on the motherboard

Part	Function
Microprocessor	Executes instructions. It sits in a socket on the board.
BIOS in ROM	Permanent start-up firmware. It runs the power-on self-test and starts loading the operating system.

Part	Function
RAM slots	Hold the memory modules that form main memory.
CMOS battery	Keeps the clock and the saved setup when the computer is unplugged.
Jumpers	Small pins and clips that select a setting, used more on older boards.
Ports	Connect external devices such as a keyboard, a display or a USB drive.
Expansion slots	Accept cards that add a function, such as a better display or a network.
System bus	The shared paths that carry addresses, data and control signals among the parts.

3.3 The microprocessor

Microprocessor. A processor built on a single integrated circuit. In a PC it is the CPU.

Inside the chip, five ideas explain how it works. The **clock** sends a regular pulse that times every step. A 3 GHz clock ticks three thousand million times a second. **Registers** are tiny storage locations inside the processor, faster than RAM, used for the instruction being executed, the address being used, and the current result. The **ALU** performs arithmetic and logical operations. The **control unit** fetches an instruction from memory, decodes it, and tells the other parts when to act. **Cache** is a small SRAM store of recently used instructions and data, so the processor waits less often for DRAM.

Outside, the chip is known by its pins. **Address pins** carry the address of the memory location the processor wants. They carry that address outward, so the address bus is unidirectional. **Data pins** carry the value being read or written, so they are bidirectional. **Control pins** carry signals such as read, write, and whether the address refers to memory or to an input-output device. **Interrupt pins** let a device ask the processor to pause and attend to it. **Power pins** supply voltage and ground.

A specification names the manufacturing technology, often in nanometres, the generation or family, the clock speed, the word size in bits (32 or 64), the width of the data bus, and the cache sizes at each level. A wider data bus moves more bits in one transfer. A wider address bus can name more locations: a 16-bit address bus can name 2 to the power 16, which is 65536 locations.

3.4 Buses

Bus. A shared set of electrical lines that carries signals from one component to another.

The system bus

Bus	Carries	Direction
Address bus	The address of a memory location or a port	Unidirectional, from the processor
Data bus	The data being read or written	Bidirectional
Control bus	Timing and command signals such as read, write and interrupt	The separate lines run in the direction each signal needs

Unidirectional means the signal travels one way. Bidirectional means it can travel either way, not both ways on the same wire at the same instant. The address leaves the processor. Data may leave or enter.

3.5 Ports

- A **serial port** sends bits one after another on a single data line. The older COM port is the classroom example. The cable can be longer. The transfer is slower than a wide parallel cable of the same era.
- A **parallel port** sends a group of bits at the same time, one bit on each wire. The older printer port is the example. It is fast over a short cable.

- **USB**, Universal Serial Bus, is the general port on a modern PC. Devices can be plugged in while the computer is on. One port can feed a hub of several devices, and the bus can supply power to small devices. Later versions of USB move data faster.
- **HDMI**, High-Definition Multimedia Interface, carries digital video and audio together to a monitor, a television or a projector.

3.6 Expansion slots

PCI, Peripheral Component Interconnect, is the common slot used to add a card such as a network card or a sound card. **EISA**, Extended Industry Standard Architecture, is an older 32-bit slot that also accepted older ISA cards. The curriculum's third name, written VISA in the acronym list, refers in this topic to the **VESA local bus** slot. That slot was tied closely to the processor bus so a video card could move data quickly. It belongs to the generation of PCs before PCI became the usual choice.

Exam point

If the question lists PCI, EISA and VISA, describe three expansion slots. In the answer you may write VESA local bus and add that the curriculum acronym is VISA. Describe the slot, not Virtual Instrument Software Architecture, which is a different use of those letters.

In the lab

Lab safety: shut the computer down, unplug the mains, and press the power button once to discharge the supply. Seat RAM only in the matching slot, with the notch lined up, until the clips close. Do this with your teacher present.

Key terms

Term	Meaning to write in an answer
Motherboard	The main circuit board that connects the processor, memory, storage and cards.
ALU	The part of the CPU that performs arithmetic and logic.
Control unit	The part of the CPU that fetches, decodes and directs instructions.
Register	A small, fast storage location inside the CPU.
Cache	Fast SRAM that holds recently used instructions and data.
Bus	A shared path of wires carrying address, data or control signals.
Port	A socket used to connect an external device.
BIOS	Firmware in ROM that starts the computer.

Bring it together

- The case, power supply, motherboard, drives, ports and cooling make the system unit.
- The processor clock, registers, ALU, control unit and cache explain the inside. Address, data, control, interrupt and power pins explain the outside.
- Address lines leave the processor. Data lines carry values in or out.
- Serial, parallel, USB and HDMI are ports. PCI, EISA and the VESA local bus are expansion slots.

Do this in the practical

- 1 Identify the processor socket, RAM slots, CMOS battery, expansion slots and one USB port on a motherboard.
- 2 With the power unplugged and a teacher present, remove and reseat a RAM module. Note the notch and the clips.

- 3 Enter the BIOS or UEFI setup and find the boot-order screen. Leave the setting as you found it unless the teacher asks for a change.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. The ALU is responsible for:

- A. Arithmetic and logic
- B. Long-term file storage
- C. Converting AC to DC
- D. Printing

2. Which bus carries the location the processor wants to read?

- A. Data bus
- B. Address bus
- C. HDMI
- D. Power supply

3. The address bus is:

- A. Bidirectional
- B. Unidirectional from the processor
- C. A printer only
- D. A type of ROM

4. Cache memory is usually built from:

- A. DRAM
- B. SRAM
- C. CD-R
- D. Magnetic tape

5. The CMOS battery:

- A. Prints documents
- B. Keeps setup and the clock when mains power is off
- C. Is the main hard disk
- D. Translates C++

6. HDMI carries:

- A. Digital video and audio
- B. Only mains electricity
- C. Only a paper signal
- D. Ultraviolet light

7. A 10-bit address bus can name:

- A. 10 locations
- B. 100 locations
- C. 1024 locations
- D. 8 locations

8. PCI is a:

- A. Type of RAM
- B. Expansion slot
- C. Keyboard layout
- D. File attribute

Answer key

Q	Answer	Why this is the answer
1	A	The arithmetic and logic unit calculates and compares.
2	B	The address bus carries the address of the location.
3	B	Addresses are issued by the processor toward memory.
4	B	Cache needs the speed of SRAM.
5	B	The battery maintains saved setup information and the time.
6	A	HDMI is a digital multimedia port for picture and sound.
7	C	Ten address lines name 2 to the power 10, which is 1024 locations.
8	B	PCI is an expansion slot for add-in cards.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Write the functions of the ALU and the control unit.

Model answer

The ALU performs arithmetic operations such as addition and logical operations such as comparison. The control unit fetches each instruction from memory, decodes it, and directs the other parts of the computer to carry it out.

Q2. Differentiate address bus and data bus.

Model answer

The address bus carries the address of a memory location and is unidirectional, from the processor. The data bus carries the data being read or written and is bidirectional. The width of the address bus limits how many locations can be named. The width of the data bus limits how many bits move in one transfer.

Q3. What is the BIOS?

Model answer

The BIOS is firmware stored in ROM on the motherboard. When the computer is switched on it runs a self-test and starts the process of loading the operating system.

Q4. Why is a heat sink fitted on the processor?

Model answer

The processor produces heat while it works. The heat sink, with its fan, conducts that heat away so the chip stays within a safe temperature.

Q5. Give one use of USB and one use of HDMI.

Model answer

USB connects general devices such as a pen drive, a printer or a keyboard, and it can supply power to small devices. HDMI connects a display and carries digital video and audio together.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Describe the main components of the system unit and the main parts found on a motherboard.

- 1 Define the system unit and name tower, desktop and all-in-one as case styles.
- 2 Explain power supply, motherboard, hard drive, combo drive, ports and cooling.
- 3 On the motherboard, give the job of the processor, BIOS, RAM slots, CMOS battery, ports, expansion slots and the system bus.
- 4 Close with the path of a program: it is stored on the drive, loaded into RAM, and executed by the processor.

Question 2. Explain the internal parts and the external pins of a microprocessor.

- 1 Define a microprocessor.
- 2 Explain clock, registers, ALU, control unit and cache.
- 3 Explain address, data, control, interrupt and power pins.
- 4 Add one specification sentence: clock speed, bit size, bus width and cache are how two processors are compared.
- 5 Give the 16-bit address example: 65536 locations.

Class XI | Unit 4 | weight 10%

Operating systems

Board weight about 10% | 10 theory periods and 10 lab periods

The operating system is the program that makes the hardware usable. It starts the computer, shares the processor, keeps files, and gives you a way to command the machine.

What you should be able to do

- Define an operating system and its objectives.
- Describe batch, multitasking, time sharing, multiprocessing, parallel, distributed and embedded systems.
- Explain the main functions, including booting and process management.
- Describe process states, and distinguish a thread from a process.
- Manage files, folders, paths, search, attributes and the control panel in a GUI.
- Use the ideas behind dxdiag and msconfig, and describe open-source and mobile systems.

Path through the lecture

- 1 Define the operating system from the user's problem: hardware is difficult to command directly.
- 2 Teach features as ways of sharing a computer, using the tables.
- 3 Walk the process-state diagram with one running program and one that is waiting for a file.
- 4 Spend the lab on real files, attributes, control panel, dxdiag and msconfig.

4.1 What an operating system is

Operating system. System software that manages hardware and software resources and provides services so that the user and the application programs can use the computer.

Its objectives are convenience, efficiency and the ability to evolve. Convenience means a user can start a program, save a file and use a printer without knowing the wiring. Efficiency means the processor, memory and disks stay busy and are shared fairly. Evolution means new hardware can be added through drivers and updates.

Systems you should recognise

System	Character
DOS	Single-user, command-line operating system used on early PCs.
Windows	A widely used graphical operating system for personal computers, with multitasking.
Mac OS	The graphical operating system used on Apple personal computers.
UNIX	A multi-user, multitasking system strong on servers and on shared machines. Many later systems took ideas from it.

Ways of working

- **Batch processing** collects jobs and runs them one after another without the user sitting at the machine for each step. A payroll run is the classic example.
- **Multitasking** keeps more than one program loaded and switches the processor among them so the user seems to have several programs at once.

- **Time sharing** gives each user a short slice of processor time. The users of a shared computer each feel they have the machine.
- **Multiprocessing** uses two or more processors in one computer system.
- **Parallel processing** splits a job across multiple processors so parts of the work proceed at the same time.
- A **distributed** system spreads the work across computers that communicate over a network.
- An **embedded** operating system runs inside a device such as a washing machine, a car controller or a POS terminal. It is built for that device.

4.2 Functions

What the operating system manages

Function	What it does
Bootting	Starts the computer and loads the operating system into RAM.
Process management	Creates, schedules and removes processes.
Memory management	Allocates RAM and keeps one process out of another's memory.
File management	Names, stores, opens and protects files and folders.
I/O management	Controls input and output devices through drivers.
Secondary storage management	Tracks free space and the location of files on disks.
Network management	Provides access to network resources.
Protection	Checks users and permissions so one person cannot freely change another's work.
Command interpreter	Reads the user's command, from a typed line or from a click, and asks the system to carry it out.

Cold booting starts from a computer that was switched off. Power reaches the board, the firmware in ROM runs a power-on self-test, finds a bootable drive, and loads the operating system. **Warm booting** restarts a computer that is already on. Modern PCs use UEFI firmware for this start-up role. The curriculum name for the start-up firmware is BIOS.

4.3 Processes

Process. A program in execution. It has its own identity, its own state, and its own address space.

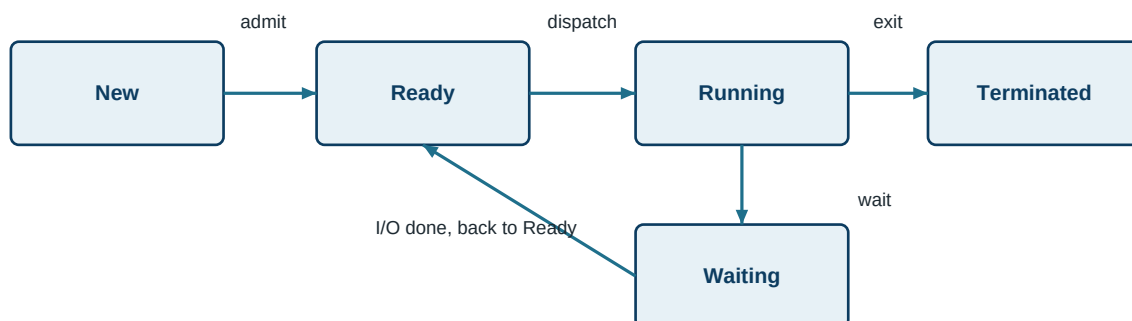


Figure 4.1 States of a process.

- **New:** the process is being created.
- **Ready:** the process has everything it needs except the processor.

- **Running:** the processor is executing its instructions.
- **Waiting or blocked:** the process cannot continue until an event finishes, often an input or output operation.
- **Terminated:** execution is finished and the operating system reclaims the resources.

A **thread** is a path of execution inside a process. Threads of one process share that process's memory. A **process** has its own memory space. **Multithreading** means one process has several threads, for example one thread drawing a page while another loads an image. **Multitasking** means the system switches among several processes. Multiprocessing means more than one processor is present.

4.4 Working in a graphical operating system

Windows presents a desktop, icons, a taskbar and windows. **Windows Explorer**, now called File Explorer, is the program that shows drives, folders and files. You create a folder, delete it, copy it, rename it, and move it by cut and paste or by dragging. A copy leaves the original in place. A move takes it to the new place.

A **drive** is a storage device with a letter, such as C. A **path** is the address of a file. An absolute path starts from the drive, for example `C:\College\CS\notes.txt`. A relative path starts from the folder you are already in. The backslash separates folder names in Windows.

Search finds a file by name, by part of a name, or by date. **Attributes** are flags stored with the file. **Read-only** refuses an ordinary save. **Hidden** keeps the file out of the ordinary listing. **System** marks a file the operating system depends on. **Archive** marks a file that should be included in the next backup. You can inspect and change attributes from the file's properties.

The **control panel** is the place where hardware and software are adjusted: display, sound, user accounts, date and time, devices, and programs. `dxdia` opens the DirectX diagnostic tool, which reports the computer's system, display and sound resources. `msconfig` opens System Configuration, which shows start-up choices and services. Change `msconfig` only when you understand the item, because a wrong boot setting can stop the machine starting normally.

4.5 Other operating systems

Linux is the usual classroom example of an operating system other than Windows. **Open-source** software is distributed with its source code. People may study it, share it, and modify it under the licence. The advantages are a low licence cost, the right to inspect the code, a community that repairs faults in public, and the freedom to adapt the system.

Android is an open-source operating system for mobile devices, built on a Linux kernel. Phones show a version number in the settings. Older classroom charts also use dessert codenames such as KitKat and Lollipop. What you need to write is that Android is a mobile operating system, it is versioned, and each version adds features and repairs.

In the lab

In the lab, create a folder CS-XI on the desktop, copy a file into it, rename the copy, make it read-only, hide it, then search for it by name. Open `dxdia` and read the operating system name and the memory size. Open `msconfig`, look at the Services tab, and close it without saving a change.

Key terms

Term	Meaning to write in an answer
Operating system	Software that manages resources and provides a way to use the computer.
Booting	The start-up sequence that loads the operating system.

Term	Meaning to write in an answer
Process	A program that is being executed.
Thread	A single path of execution inside a process.
Multitasking	Sharing one processor among several tasks by switching.
Time sharing	Giving each user a slice of processor time.
Path	The location of a file, written as drive and folders.
Open source	Software whose source code may be studied and modified under its licence.

Bring it together

- The operating system makes hardware usable and shares it safely.
- Batch, interactive multitasking, time sharing, multiprocessing, parallel, distributed and embedded are different ways of organising that work.
- A process moves through new, ready, running, waiting and terminated.
- Files live at paths and carry attributes. The control panel, dxdiag and msconfig are the Windows tools named in this unit.

Do this in the practical

- 1 Show the desktop, File Explorer, a drive and a path.
- 2 Create, rename, copy, move and delete a folder. Search for the file afterwards.
- 3 Set and clear the read-only and hidden attributes.
- 4 Open control panel, dxdiag and msconfig. Record the operating system name and the installed RAM. Do not save a change in msconfig.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. A program in execution is a:

- A. File attribute
- B. Process
- C. Port
- D. Nibble

2. Which state means the process is waiting for the processor?

- A. Terminated
- B. Ready
- C. New only
- D. ROM

3. Time sharing is used so that:

- A. Each user gets a slice of CPU time
- B. Files are compressed
- C. ROM is erased
- D. A plotter draws faster

4. An embedded OS is found in:

- A. A device built for a specific product
- B. A compiler only
- C. A nibble
- D. A barcode definition

5. The hidden attribute:

- A. Speeds up the ALU
- B. Keeps a file out of the ordinary listing
- C. Converts AC to DC
- D. Is a network topology

6. dxdiag is used to:

- A. Diagnose DirectX system, display and sound information
- B. Erase EPROM with ultraviolet light
- C. Design a bus topology
- D. Compile C++

7. Threads of the same process:

- A. Must each have a separate hard disk
- B. Share that process's memory
- C. Are always stored in ROM
- D. Replace the power supply

8. Open-source software allows the user to:

- A. Study and modify the source under the licence
- B. Remove the need for a processor
- C. Disable every attribute
- D. Turn DRAM into a printer

Answer key

Q	Answer	Why this is the answer
1	B	A process is a program that is running.
2	B	A ready process can run as soon as the scheduler dispatches it.
3	A	Time sharing switches the CPU among users quickly.
4	A	Embedded systems run inside dedicated devices.
5	B	Hidden is a file attribute that suppresses the normal display.
6	A	dxdiag reports the multimedia and system resources Windows can see.
7	B	Threads are lightweight paths of execution inside one process address space.
8	A	The source is available for study and modification under its licence.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Define an operating system.

Model answer

An operating system is system software that manages hardware and software resources and provides services through which the user and application programs use the computer.

Q2. Differentiate multitasking and multiprocessing.

Model answer

Multitasking switches one processor among several tasks so they appear to run together. Multiprocessing uses two or more processors in the computer. A machine can do both: several processors, each shared among tasks.

Q3. Differentiate a process and a thread.

Model answer

A process is a running program with its own address space. A thread is one path of execution inside a process. Threads of a process share its memory. Creating a thread costs less than creating another process.

Q4. List the five states of a process.

Model answer

New, ready, running, waiting (blocked), and terminated. A new process is admitted to ready. The scheduler dispatches a ready process to running. It may wait for input or output, return to ready when that finishes, or terminate.

Q5. What is a file path? Give one example.

Model answer

A path is the location of a file in the folder tree. An absolute Windows path includes the drive, for example C:\College\CS\notes.txt.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain the main functions of an operating system.

- 1 Define the operating system and its objectives: convenience, efficiency, evolution.
- 2 Explain booting, process management, memory management and file management.
- 3 Explain I/O, secondary storage, network, protection and the command interpreter.
- 4 Give one example for protection, such as a user account and a read-only file.

Question 2. Describe process states and distinguish multithreading from multitasking.

- 1 Define a process.
- 2 Draw or describe new, ready, running, waiting and terminated, with the reason for each move.
- 3 Define a thread and say that threads share a process's memory.
- 4 Multithreading is many threads in one process. Multitasking is many processes sharing the processor. Multiprocessing is many processors.

Class XI | Unit 5 | weight 20%

Programming in C++

Board weight about 20% | 20 theory periods and 20 lab periods

This is the heaviest unit in Class XI. You will write complete C++ programs that calculate, decide, repeat, and call functions. Every program in this lecture was written to compile and run.

What you should be able to do

- Use C++ data types, operators, and cin and cout, including cascading.
- Write selection with if, if-else, else-if, switch and nested if.
- Write for, while, do-while and nested loops, and use break, continue and goto correctly.
- Declare, define and call functions. Explain scope, parameters, inline functions and exit.
- Pass constants, pass by value and pass by reference. Use default arguments and return values.
- Overload functions by the number and types of arguments.

Path through the lecture

- 1 Type the calculator on the first lab day. Do not stay on the board for the whole period.
- 2 Add decisions, then loops, then functions. Each lab program below is one of those steps.
- 3 Teach pass by reference with the before-and-after output on the screen.
- 4 End with overloading, and say clearly that return type alone does not overload a function.

5.1 Data, operators and streams

A C++ program for this course starts with the iostream header and a main function. cin reads from the keyboard. cout writes to the screen. The insertion operator << sends a value out. The extraction operator >> brings a value in. Cascading means chaining those operators in one statement.

Types you will use

Type	Holds	Example
int	Whole numbers	int marks = 76;
float	Numbers with a fractional part	float average = 72.5;
double	A fractional number kept more precisely	double tax = 0.05;
char	One character	char grade = 'A';
bool	true or false	bool pass = true;
void	No value, used for a function that only does work	void drawStars(int n);

Operators

Family	Operators	Meaning
Arithmetic	+ - * / %	Add, subtract, multiply, divide, remainder
Assignment	= += -=	Store a value, or change the stored value
Relational	== != > < >= <=	Compare two values. The result is true or false
Logical	&& !	And, or, not
Increment	++ --	Add 1 or subtract 1

Family	Operators	Meaning
Conditional	?:	Choose one of two values

Integer division drops the fraction: $7 / 2$ is 3. The remainder operator gives what is left: $7 \% 2$ is 1. Use `==` to compare and `=` to assign. Confusing those two is the most common logic error in a condition.

Program 5.1 Calculator, with cascading output

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two integers: ";
    cin >> a >> b;
    cout << "Sum = " << a + b << endl;
    cout << "Difference = " << a - b << endl;
    cout << "Product = " << a * b << endl;
    if (b != 0)
        cout << "Quotient = " << a / b << endl;
    else
        cout << "Cannot divide by zero" << endl;
    return 0;
}
```

The division line is guarded. Division by zero is undefined, so the program tests the divisor first. Sample run: input 12 3 prints sum 15, difference 9, product 36 and quotient 4.

5.2 Decisions

`if` runs a block only when the condition is true. `if-else` chooses one of two blocks. `else if` chains several conditions and the first true condition wins. A nested `if` is an `if` inside another `if`. `switch` compares one integer or character with several constant cases. End each case with `break`, otherwise execution falls into the next case. `default` runs when no case matches.

Program 5.2 A mark sheet using else-if

```
#include <iostream>
using namespace std;

int main() {
    int marks;
    cout << "Enter marks out of 100: ";
    cin >> marks;
    cout << "Grade: ";
    if (marks < 0 || marks > 100)
        cout << "Invalid" << endl;
    else if (marks >= 80)
        cout << "A" << endl;
    else if (marks >= 70)
        cout << "B" << endl;
    else if (marks >= 60)
        cout << "C" << endl;
    else if (marks >= 50)
        cout << "D" << endl;
    else
        cout << "Fail" << endl;
    return 0;
}
```

These grade bands are a classroom exercise so the selection is easy to test. They are not the board's own grading rules. Input 76 prints grade B. Input 110 prints Invalid because the mark is outside 0 to 100.

Program 5.3 Largest of three, using nested decisions

```
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cout << "Enter three integers: ";
    cin >> a >> b >> c;
    int largest = a;
    if (b > largest)
        largest = b;
    if (c > largest)
        largest = c;
    cout << "Largest = " << largest << endl;
    return 0;
}
```

Program 5.4 switch for the day of the week

```
#include <iostream>
using namespace std;

int main() {
    int day;
    cout << "Enter a day number (1 to 7): ";
    cin >> day;
    switch (day) {
        case 1: cout << "Monday" << endl; break;
        case 2: cout << "Tuesday" << endl; break;
        case 3: cout << "Wednesday" << endl; break;
        case 4: cout << "Thursday" << endl; break;
        case 5: cout << "Friday" << endl; break;
        case 6: cout << "Saturday" << endl; break;
        case 7: cout << "Sunday" << endl; break;
        default: cout << "Invalid day" << endl;
    }
    return 0;
}
```

5.3 Loops and jumps

A `for` loop is the natural choice when you know how many times to repeat. A `while` loop tests the condition before the body, so the body may never run. A `do-while` runs the body once and then tests, so it is the right loop for a menu. A nested loop has a loop inside a loop. The inner loop finishes its repetitions for each step of the outer loop.

Program 5.5 Prime or composite

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a positive integer: ";
    cin >> n;
    if (n <= 1) {
        cout << n << " is neither prime nor composite" << endl;
        return 0;
    }
    bool isPrime = true;
    for (int i = 2; i <= n / 2; i++) {
        if (n % i == 0) {
            isPrime = false;
            break;
        }
    }
    if (isPrime)
        cout << n << " is prime" << endl;
    else
        cout << n << " is composite" << endl;
    return 0;
}
```

The number 1 is neither prime nor composite. A prime number is greater than 1 and has no factor other than 1 and itself. The loop tests divisors from 2 up to $n/2$. If any divisor divides exactly, the number is composite and break leaves the loop.

Program 5.6 Four digits on four lines

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a four-digit number: ";
    cin >> n;
    if (n < 1000 || n > 9999) {
        cout << "Enter a four-digit number" << endl;
        return 0;
    }
    cout << n / 1000 << endl;
    cout << (n / 100) % 10 << endl;
    cout << (n / 10) % 10 << endl;
    cout << n % 10 << endl;
    return 0;
}
```

Integer division by 1000 isolates the thousands digit. The remainder after division by 10 isolates the last digit. Input must be from 1000 to 9999.

Program 5.7 Factors of a number

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a positive integer: ";
    cin >> n;
    cout << "Factors: ";
    for (int i = 1; i <= n; i++) {
        if (n % i == 0)
            cout << i << " ";
    }
    cout << endl;
    return 0;
}
```

Program 5.8 A menu with do-while

```
#include <iostream>
using namespace std;

int main() {
    int choice, n;
    do {
        cout << "1. Square a number" << endl;
        cout << "0. Exit" << endl;
        cout << "Choice: ";
        cin >> choice;
        if (choice == 1) {
            cout << "Number: ";
            cin >> n;
            cout << "Square = " << n * n << endl;
        }
    } while (choice != 0);
    cout << "Done" << endl;
    return 0;
}
```

`break` leaves the nearest loop or switch immediately. `continue` skips the rest of this repetition and goes to the next test. `goto` jumps to a label. It is in the curriculum so you can recognise it. Do not use it in ordinary programs: a loop or a function states the intention more clearly, and a `goto` makes a long program hard to follow.

Program 5.9 `break`, `continue` and a labelled `goto`

```
#include <iostream>
using namespace std;

int main() {
    cout << "continue skips 3: ";
    for (int i = 1; i <= 5; i++) {
        if (i == 3)
            continue;
        cout << i << " ";
    }
    cout << endl;
    for (int i = 1; i <= 5; i++) {
        if (i == 3) {
            cout << "Stopped at 3" << endl;
            break;
        }
    }
    int n = 1;
print_line:
    if (n == 1)
        cout << "goto reached the label" << endl;
    return 0;
}
```

5.4 Functions

A function is a named block of work. **Predefined** functions come with the language or its library, such as `sqrt` in `cmath`. **User-defined** functions are the ones you write. Functions let you name a task, test it once, and call it from several places. A large program becomes a set of small ones.

The **signature** is the function's name, its parameter list, and its return type. A **prototype** declares the function before `main`. The **definition** contains the body. A **call** uses the function. Formal parameters are the names in the definition. Actual parameters are the values sent at the call.

A **local variable** exists only inside the block where it is declared. A **global variable** is declared outside functions and can be used by all of them; use it rarely, because any function can change it. A **static local variable** keeps its value between calls to that function.

An **inline** function is a short function that the compiler may place at the point of the call, avoiding a call for a one-line calculation. `exit`, from `cstdlib`, ends the whole program at once. A `return` statement leaves only the current function and can carry a value back to the caller.

Program 5.10 A function that draws a triangle of stars

```
#include <iostream>
using namespace std;

void drawStars(int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++)
            cout << "*";
        cout << endl;
    }
}

int main() {
    int size;
    cout << "Enter size: ";
    cin >> size;
    drawStars(size);
    return 0;
}
```

Program 5.11 Return the larger of two arguments

```
#include <iostream>
using namespace std;

int larger(int a, int b) {
    if (a > b)
        return a;
    return b;
}

int main() {
    int x, y;
    cout << "Enter two integers: ";
    cin >> x >> y;
    cout << "Larger = " << larger(x, y) << endl;
    return 0;
}
```

5.5 Passing arguments

- **By value:** the function receives a copy. Changing the parameter does not change the caller's variable.
- **By reference:** the parameter is an alias for the caller's variable, written with &. Changing it changes the original.
- A **const parameter** is a promise not to modify that parameter inside the function.
- A **default argument** is a value used when the caller omits that argument. Defaulted parameters go at the end of the list.

Program 5.12 Value, reference, const, inline and a default argument

```
#include <iostream>
using namespace std;

void byValue(int n) { n = 99; }

void byReference(int &n) { n = 99; }

void showConst(const int n) { cout << "Constant parameter: " << n << endl; }

inline int square(int n) { return n * n; }

int add(int a, int b = 10) { return a + b; }

int main() {
    int a = 10;
    byValue(a);
    cout << "After by value: " << a << endl;
    byReference(a);
    cout << "After by reference: " << a << endl;
    showConst(7);
    cout << "square(6) = " << square(6) << endl;
    cout << "add(5) = " << add(5) << endl;
    cout << "add(5, 3) = " << add(5, 3) << endl;
    return 0;
}
```

Read the output of Program 5.12 carefully. After the call by value the variable is still 10. After the call by reference it is 99. add(5) uses the default second argument and returns 15. add(5, 3) returns 8. square(6) returns 36.

Program 5.13 exit ends the program

```

#include <iostream>
#include <cstdlib>
using namespace std;

int visits = 0;

void leave() {
    cout << "Goodbye" << endl;
    exit(0);
}

int main() {
    int n;
    static int calls = 0;
    calls++;
    visits++;
    cout << "Enter a positive mark, or -1 to stop: ";
    cin >> n;
    if (n < 0)
        leave();
    cout << "Mark stored. calls = " << calls << endl;
    cout << "visits = " << visits << endl;
    return 0;
}

```

5.6 Overloading

Function overloading. Two or more functions with the same name and different parameter lists, in the same scope. The compiler chooses the function from the arguments at the call.

You may overload on the number of arguments or on the types of the arguments. You may not overload on the return type alone. Two functions called `sum` that both take two ints and differ only because one returns int and one returns double are rejected. The call would not tell the compiler which one you meant.

Program 5.14 Overloading on number and type of arguments

```

#include <iostream>
using namespace std;

int sum(int a, int b) { return a + b; }

int sum(int a, int b, int c) { return a + b + c; }

double sum(double a, double b) { return a + b; }

int main() {
    cout << "int sum = " << sum(3, 4) << endl;
    cout << "three sum = " << sum(3, 4, 5) << endl;
    cout << "double sum = " << sum(2.5, 3.0) << endl;
    return 0;
}

```

Exam point

In a board program, write the include line, using `namespace std;`, a complete main, and matching braces. Initialise variables before you read them in an expression. Test the program with the sample in the question and with one awkward value, such as 0 or 1.

Key terms

Term	Meaning to write in an answer
Operator	A symbol that asks for an operation on one or more values.

Term	Meaning to write in an answer
Cascading	Chaining << or >> in a single input or output statement.
Loop	A statement that repeats a block while a condition holds.
Prototype	A declaration of a function's name, return type and parameters.
Formal parameter	The parameter name written in the function definition.
Actual parameter	The value or variable supplied at the call.
Pass by reference	Passing a variable so the function can change the original.
Overloading	Same function name, different parameter lists.

Bring it together

- cin and cout, with << and >>, are the streams for this course.
- if, else-if and switch make decisions. for, while and do-while repeat. break and continue alter one loop.
- A function has a prototype, a definition and a call. It can return a value.
- Pass by value protects the original. Pass by reference changes it. Defaults fill in missing arguments.
- Overload by arguments, never by return type alone.

Do this in the practical

- 1 Build the calculator, the mark sheet, the prime test, the four-digit split and the factor list.
- 2 Write drawStars and larger as separate functions and call them from main.
- 3 Show, with printed output, that pass by value does not change the original and pass by reference does.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. Which operator finds the remainder?

- A. /
- B. %
- C. ==
- D. &&

2. A do-while loop:

- A. Never runs the body
- B. Tests after the body, so the body runs at least once
- C. Can only count down
- D. Replaces a function

3. break inside a loop:

- A. Skips to the next repetition
- B. Leaves the loop
- C. Deletes the file
- D. Overloads the function

4. A default argument must:

- A. Be the first parameter
- B. Appear at the end of the parameter list
- C. Be a pointer
- D. Be global

5. Which pair can be overloaded together?

- A. `int f(int)` and `double f(int)`
- B. `int f(int)` and `int f(int, int)`
- C. Two identical functions
- D. `int f(int)` and `int f(int)`

6. Pass by value:

- A. Sends a copy
- B. Always changes the caller's variable
- C. Needs the `&` symbol on the parameter
- D. Ends the program

7. The correct way to compare two ints for equality is:

- A. `a = b`
- B. `a == b`
- C. `a := b`
- D. `a eq b`

8. `exit(0)` :

- A. Leaves only the current loop
- B. Terminates the program
- C. Draws a star
- D. Opens a file

Answer key

Q	Answer	Why this is the answer
1	B	The <code>%</code> operator yields the remainder of integer division.
2	B	The test is at the bottom, so one execution happens before the test.
3	B	<code>break</code> terminates the nearest loop or switch.
4	B	Defaults are trailing parameters so the call stays unambiguous.
5	B	The parameter lists differ in the number of arguments. Return type alone is not enough.
6	A	The function works on a copy, so the original is unchanged.
7	B	<code>==</code> compares. A single <code>=</code> assigns.
8	B	<code>exit</code> ends the entire program. <code>return</code> leaves only the current function.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. What is cascading of the output stream? Write one statement.

Model answer

Cascading means using << more than once in a single statement. Example: `cout << "Sum = " << a + b << endl;` The text and the number are sent out by the same statement.

Q2. Differentiate while and do-while.

Model answer

while tests the condition before the body, so the body may not run at all. do-while runs the body and then tests, so the body runs at least once. do-while suits a menu that must appear before the user chooses.

Q3. Differentiate formal and actual parameters.

Model answer

Formal parameters are the names in the function heading, such as `int a` and `int b` in `int larger(int a, int b)`. Actual parameters are the values at the call, such as `larger(x, y)`. The actual values are passed into the formal parameters.

Q4. Why can two functions not be overloaded by return type alone?

Model answer

The compiler selects the overloaded function from the arguments at the call. If both functions take the same arguments, the call does not show which function is wanted, even if the return types differ. The program is rejected.

Q5. What does a static local variable do?

Model answer

A static local variable is created once and keeps its value after the function returns. The next call sees the value left by the previous call. An ordinary local variable is created again on each call.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain selection and iteration structures in C++, with the job of each.

- 1 Define selection as choosing a path, and name if, if-else, else-if, nested if and switch. Mention break and default.
- 2 Give a one-line situation for else-if, such as a grade, and for switch, such as a menu number.
- 3 Define iteration and compare for, while and do-while, including how many times the body can run.
- 4 Explain nested loops with the star triangle: the outer loop is the row, the inner loop prints the stars.
- 5 Name break and continue and give the effect of each.

Question 2. Explain user-defined functions, parameter passing and overloading.

- 1 Give the advantages: a named task, reuse, and a smaller main.
- 2 Show prototype, definition and call, and define the signature.
- 3 Contrast pass by value and pass by reference with the & sign and the effect on the original variable.
- 4 Explain default arguments and the return statement.
- 5 Define overloading and state the rule about argument lists and return types. Give sum of two ints and sum of three ints as the example.

Class XI | Unit 6 | weight 15%

Arrays, strings and structures

Board weight about 15% | 25 theory periods and 25 lab periods

An array stores many values of one type under one name. A string stores text. A structure stores related values of different types as one record. Most of the Class XI practical journal lives in this unit.

What you should be able to do

- Define an array, its name, size and index, and initialise arrays of different types.
- Access an element, traverse an array, and use sizeof.
- Declare, initialise and process a two-dimensional array.
- Use C strings with the terminator, and use strlen, strcat, strcpy, strcmp and substr.
- Declare a structure, create a variable, and read and display its members.

Path through the lecture

- 1 Draw five boxes with indexes 0 to 4 before any syntax.
- 2 Traverse, search, sort and average on the same small list so the loops look familiar.
- 3 Treat a string as an array of char with a hidden zero at the end.
- 4 Close with one employee record. That is the bridge to classes in Class XII.

6.1 Arrays

Array. A collection of elements of the same data type, stored in consecutive memory locations and referred to by one name and an index.

The **name** is the identifier, such as marks. The **size** is how many elements were reserved. The **index** is the position of one element. In C++ the first index is 0. An array of 5 elements uses indexes 0, 1, 2, 3 and 4. There is no element at index 5. Using one is a serious error.

You can initialise an array where you declare it. `int marks[5] = {10, 20, 30, 40, 50};` sets every element. `char grades[3] = {'A', 'B', 'C'};` is an array of characters, which is not yet a C string because there is no terminator. Access one element with the index in brackets: `marks[2]` is 30.

`sizeof(marks)` gives the storage of the whole array in bytes. `sizeof(marks[0])` gives the storage of one element. Their quotient is the number of elements. On a typical PC an int is 4 bytes, so five ints occupy 20 bytes.

Program 6.1 Initialise, access, traverse and measure an array

```
#include <iostream>
using namespace std;

int main() {
    int marks[5] = {10, 20, 30, 40, 50};
    cout << "Element at index 2 = " << marks[2] << endl;
    cout << "Bytes = " << sizeof(marks) << endl;
    cout << "Elements = " << sizeof(marks) / sizeof(marks[0]) << endl;
    cout << "Traverse: ";
    for (int i = 0; i < 5; i++)
        cout << marks[i] << " ";
    cout << endl;
    return 0;
}
```

Traversal means visiting every element once. The for-loop from 0 to size-1 is the pattern you will reuse for search, total and display.

6.2 Two-dimensional arrays

Two-dimensional array. An array whose elements are arranged in rows and columns. An element needs a row index and a column index.

`int table[2][3]` has 2 rows and 3 columns, so 6 ints. In memory the rows are stored one after another. The initialiser lists the rows in order. `table[1][2]` is row 1, column 2, counting from zero, which is the last element of the second row.

Program 6.2 A 2 by 3 array

```
#include <iostream>
using namespace std;

int main() {
    int table[2][3] = {{1, 2, 3}, {4, 5, 6}};
    for (int r = 0; r < 2; r++) {
        for (int c = 0; c < 3; c++)
            cout << table[r][c] << " ";
        cout << endl;
    }
    cout << "Row 1, column 2 = " << table[1][2] << endl;
    return 0;
}
```

Addition of two matrices is element by element: each sum element is $a[i][j] + b[i][j]$. The matrices must have the same number of rows and the same number of columns. Multiplication of a 3 by 3 matrix needs three nested loops. The element in row i and column j of the product is the sum of $a[i][k] * b[k][j]$ as k runs across the row and down the column.

Program 6.3 Add and multiply 3 by 3 matrices

```
#include <iostream>
using namespace std;

int main() {
    int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int b[3][3] = {{1, 0, 0}, {0, 1, 0}, {0, 0, 1}};
    int sum[3][3], product[3][3];

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++)
            sum[i][j] = a[i][j] + b[i][j];
    }
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            product[i][j] = 0;
            for (int k = 0; k < 3; k++)
                product[i][j] += a[i][k] * b[k][j];
        }
    }
    cout << "Sum row0: " << sum[0][0] << " " << sum[0][1]
        << " " << sum[0][2] << endl;
    cout << "Product row0: " << product[0][0] << " "
        << product[0][1] << " " << product[0][2] << endl;
    cout << "Product row2: " << product[2][0] << " "
        << product[2][1] << " " << product[2][2] << endl;
    return 0;
}
```

In Program 6.3 the second matrix is an identity matrix, so the product equals the first matrix. The first row of the sum is 2, 2, 3 because the identity adds 1 only to the diagonal.

6.3 Searching, sorting and the average

A linear search looks at each element until it finds the key or reaches the end. Its place in the answer is the index, or a clear 'not found'. Bubble sort repeatedly walks the list and swaps a pair that is out of order. After each outer pass, one more value has settled at the end. For names stored as C strings, the comparison is `strcmp`, and the swap copies whole strings with `strcpy`.

Program 6.4 Bubble sort, name sort and linear search

```
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    int a[] = {5, 1, 4, 2};
    int n = 4;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
    cout << "Sorted: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;

    char names[3][10] = {"Sara", "Ali", "Hina"};
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2 - i; j++) {
            if (strcmp(names[j], names[j + 1]) > 0) {
                char temp[10];
                strcpy(temp, names[j]);
                strcpy(names[j], names[j + 1]);
                strcpy(names[j + 1], temp);
            }
        }
    }
    cout << "Names: ";
    for (int i = 0; i < 3; i++)
        cout << names[i] << " ";
    cout << endl;

    int key;
    cout << "Search for: ";
    cin >> key;
    int index = -1;
    for (int i = 0; i < n; i++) {
        if (a[i] == key)
            index = i;
    }
    if (index >= 0)
        cout << "Found at index " << index << endl;
    else
        cout << "Not found" << endl;
    return 0;
}
```

Program 6.5 Average of a list

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "How many numbers? ";
    cin >> n;
    int sum = 0;
    for (int i = 0; i < n; i++) {
        int value;
        cin >> value;
        sum += value;
    }
    cout << "Average = " << sum / n << endl;
    return 0;
}
```

Program 6.5 uses integer division because the total and the count are ints. The average of 10, 20, 30 and 40 is 25 exactly. If you need a fractional average, store the total in a double.

6.4 Strings

C string. An array of char that ends with the null character `\0`. The null character is the terminator. It marks the end and is not part of the text.

`char name[20] = "Ali";` stores A, l, i, and then `\0`. The size 20 is the room available, not the length of the text. `strlen(name)` counts characters before the terminator, so the length is 3, not 20 and not 4.

String operations in this unit

Operation	What it does	Care
<code>strlen(s)</code>	Counts characters before the terminator	The terminator is not counted
<code>strcpy(dest, src)</code>	Copies src into dest, including the terminator	dest must be large enough
<code>strcat(dest, src)</code>	Joins src onto the end of dest	dest must have room for both strings
<code>strcmp(a, b)</code>	Compares a and b. The result is 0 when they are equal	A negative result means a comes first
<code>s.substr(pos, n)</code>	From a <code>std::string</code> , takes n characters starting at pos	This is a member of the string class, not a C-string function

The curriculum name `substr` belongs to the C++ string class. The C functions `strlen`, `strcpy`, `strcat` and `strcmp` belong to `cstring` and work on char arrays. Keep the two styles apart. Do not pass a `std::string` to `strcpy`.

Program 6.6 Length, reverse, join, compare and substr

```

#include <iostream>
#include <cstring>
#include <string>
using namespace std;

int main() {
    char name[40];
    cout << "Enter a single-word name: ";
    cin >> name;
    cout << "Length = " << strlen(name) << endl;
    cout << "Reverse = ";
    for (int i = strlen(name) - 1; i >= 0; i--)
        cout << name[i];
    cout << endl;

    char first[20] = "BIEK";
    char second[20] = "Karachi";
    char joined[40];
    strcpy(joined, first);
    strcat(joined, " ");
    strcat(joined, second);
    cout << "Joined = " << joined << endl;
    if (strcmp(first, second) < 0)
        cout << "BIEK comes before Karachi" << endl;

    string word = "Computer";
    cout << "Part = " << word.substr(0, 3) << endl;
    return 0;
}

```

Reversing a name is a loop from the last character down to index 0. For Ayesha the length is 6 and the reverse is ahseyA. strcpy followed by two strcat calls builds BIEK Karachi. substr(0, 3) on Computer gives Com.

6.5 Structures

Structure. A user-defined type that groups members of different types under one name. Each member has its own name.

The struct block declares the type. It does not by itself create a variable. `Employee emp;` creates a variable. Members are reached with a dot: `emp.salary`. A structure can be initialised with a brace list in the same order as the members. An array of structures is a table of records, which is the idea you will meet again in files and in databases.

Program 6.7 An employee record

```
#include <iostream>
#include <cstring>
using namespace std;

struct Employee {
    char name[30];
    char designation[20];
    int salary;
};

int main() {
    Employee emp;
    cout << "Enter name, designation and salary: ";
    cin >> emp.name >> emp.designation >> emp.salary;
    cout << "Name: " << emp.name << endl;
    cout << "Designation: " << emp.designation << endl;
    cout << "Salary: " << emp.salary << endl;
    return 0;
}
```

This version reads a single-word name with `cin >>`. A name with a space needs `getline`, and mixing `getline` with `>>` needs care about the leftover newline. For the board, a single-word name is the expected program unless the question asks for full names.

Exam point

When a 3-mark question says 'declare a structure for a book', write the struct with the members they name, then declare one variable, then show one member being assigned. Three short pieces of code answer the question.

Key terms

Term	Meaning to write in an answer
Index	The position of an element. The first index in C++ is 0.
Traversal	Visiting every element of an array once.
Null character	The '\0' terminator that ends a C string.
strlen	Returns the number of characters before the terminator.
strcmp	Compares two C strings. Zero means they are equal.
Structure	A group of related members that may have different types.
Member	One named field inside a structure.

Bring it together

- An array is one name, one type, many elements, indexes from 0.
- `sizeof` gives bytes. Divide by the size of one element to get the count.
- Two indexes select a row and a column. Matrix addition matches positions. Multiplication combines a row with a column.
- A C string ends with `\0`. `strlen`, `strcpy`, `strcat` and `strcmp` are the C functions. `substr` belongs to `std::string`.
- A structure groups different types. The dot operator reaches a member.

Do this in the practical

- 1 Sort a numeric list and a short list of names. Search for a value and print its index.
- 2 Average a list. Add and multiply two 3 by 3 matrices.
- 3 Display a name in reverse using `strlen`.

- 4 Create an Employee structure with name, designation and salary, then display one employee.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. The index of the first element of a C++ array is:

- A. 1
- B. 0
- C. -1
- D. The size of the array

2. `int a[4]` reserves indexes:

- A. 1 to 4
- B. 0 to 3
- C. 0 to 4
- D. 4 to 8

3. `strlen` on a C string counts:

- A. The array capacity, including unused room
- B. Characters before the terminator
- C. Only vowels
- D. The number of arrays in the program

4. `strcmp` returns 0 when:

- A. The strings differ
- B. The strings are equal
- C. The first string is empty always
- D. A file is closed

5. `strcpy(dest, src)` copies:

- A. `src` into `dest`
- B. `dest` into `src`
- C. Only the first character, and stops
- D. A structure member into ROM

6. A structure is used when:

- A. All values must be the same type and nothing else is allowed
- B. Related members of different types form one record
- C. Only one `int` is needed
- D. A bus topology is drawn

7. `table[1][2]` in an array declared `[2][3]` is:

- A. Always out of range
- B. The last column of the second row
- C. The first element
- D. A string function

8. The member operator for a structure variable is:

- A. `->`
- B. `.`
- C. `::`
- D. `&`

Answer key

Q	Answer	Why this is the answer
1	B	C++ arrays are numbered from zero.
2	B	Four elements use indexes 0, 1, 2 and 3.
3	B	The terminator marks the end and is not included in the length.
4	B	A zero result means the two C strings match.
5	A	The destination is the first argument.
6	B	Structures group different types. Arrays group one type.
7	B	Row index 1 is the second row. Column index 2 is the third column.
8	B	A structure variable uses the dot. The arrow is used with a pointer to a structure.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Define an array. What are its name, size and index?

Model answer

An array is a collection of elements of the same type in consecutive memory, addressed by one name and an index. The name identifies the collection. The size is the number of elements reserved. The index selects one element. In C++ the first index is 0.

Q2. What does sizeof tell you about an array?

Model answer

sizeof(array) is the number of bytes occupied by the whole array. Dividing by sizeof(array[0]) gives the number of elements. It does not return the length of a C string; strlen does that.

Q3. Why does a C string need '\0'?

Model answer

The terminator marks where the text ends inside a char array that may be larger than the text. strlen, strcpy and the output of cout stop at that character. Without it, a string function reads on into neighbouring memory.

Q4. Differentiate an array and a structure.

Model answer

An array holds many elements of one type, selected by an index. A structure holds named members that may have different types, selected by the member name. A class register of marks can be an array. One student's name, class and fee can be a structure.

Q5. Write the statement that joins two C strings.

Model answer

Place the first string in a char array large enough for both, then call strcat(first, second). strcat appends the second string and its terminator onto the first. strcpy is the copy, not the join.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain one-dimensional and two-dimensional arrays, with initialisation and access.

- 1 Define an array and the terms name, size and index. State that the first index is 0.
- 2 Show a declaration with an initialiser and one access, such as marks[2].
- 3 Show a traversal loop and the sizeof quotient.
- 4 Define a two-dimensional array as rows and columns. Show int t[2][3] and the element t[1][2].
- 5 State the rule for matrix addition and the three-loop pattern for multiplication.

Question 2. Explain C strings and write the purpose of strlen, strcpy, strcat, strcmp and substr.

- 1 Define a C string as a char array ending in \0. Distinguish capacity from length.
- 2 Give strlen, strcpy, strcat and strcmp with one sentence and one caution each.
- 3 Show substr on a std::string and say it is not a cstring function.
- 4 Describe the reverse-name loop from length-1 down to 0.

Class XI | Unit 7 | weight 15%

Computer communication and networks

Board weight about 15% | 25 theory periods

A network is computers and other devices joined so they can exchange data. This lecture names the parts of a communication, the directions data can travel, the media, the shapes of a network, and the layered models used to describe it.

What you should be able to do

- Describe a network, its advantages, and the five communication components.
- Define simplex, half duplex and full duplex.
- Compare guided and unguided media.
- Describe LAN and WAN, and the bus, star, ring, mesh, tree and hybrid topologies.
- Name ISO, IEEE, ITU and ASCII, and describe the OSI, TCP/IP and Internet models.

Path through the lecture

- 1 Start from a message the class actually sends, and name sender, receiver, medium, message and protocol on that example.
- 2 Teach direction with a television, a walkie-talkie and a telephone.
- 3 Draw bus, star and ring. Describe mesh, tree and hybrid from the drawings.
- 4 Build the OSI stack from the physical wire up to the program the user touches. Place TCP/IP beside it.

7.1 A network and its parts

Computer network. A collection of computers and other devices connected so that they can communicate and share resources.

A college network shares printers, shares files, lets staff send messages, keeps a backup in one place, and gives many computers a path to the internet. The sharing has to be organised, and the cable or wireless link has to be protected and maintained.

Five components of communication

Component	In a college email
Sender	The computer of the person who writes the message
Receiver	The computer of the person who reads it
Medium	The cable or the wireless link that carries the signal
Message	The mail itself
Protocol	The agreed rules, such as the rules that format and deliver mail

Protocol. A set of rules that communicating devices follow so that a message sent by one can be understood by the other.

7.2 Direction of data

Communication modes

Mode	Direction	Example
Simplex	One direction only	A keyboard sending to the computer, or a television broadcast

Mode	Direction	Example
Half duplex	Both directions, one at a time	A walkie-talkie
Full duplex	Both directions at the same time	A telephone conversation, and ordinary Ethernet

7.3 Transmission media

Guided medium. A physical path that contains the signal: a wire or a fibre.

Unguided medium. A wireless path. The signal travels through the air or space.

- **Twisted pair** is the ordinary copper LAN cable. Wires are twisted to reduce interference. Shielded twisted pair adds a foil or braid. Unshielded twisted pair is the common Ethernet cable.
- **Coaxial cable** has a central copper core, insulation, a shield and a jacket. It resists interference and was widely used for cable television and older networks.
- **Fibre optic** cable carries pulses of light in a glass or plastic core. It carries data over long distances with little electrical interference and with high capacity.
- **Radio waves** travel in many directions and pass through ordinary walls. They carry Wi-Fi and many mobile signals.
- **Microwaves** travel in a straight line between antennas and are used for long point-to-point links. The antennas need a clear path.
- **Infrared** is a short-range light signal, used by some remote controls. It does not pass through walls.

7.4 Types and topologies

A **LAN**, local area network, covers a room, a lab or a building and is owned by one organisation. A **MAN**, metropolitan area network, covers a town or a city. A **WAN**, wide area network, covers a large region or the world. The internet is the WAN students use every day. The unit list concentrates on LAN and WAN. MAN is the standard type between them and is in the curriculum's own acronyms.

Topology. The arrangement of devices and links in a network.

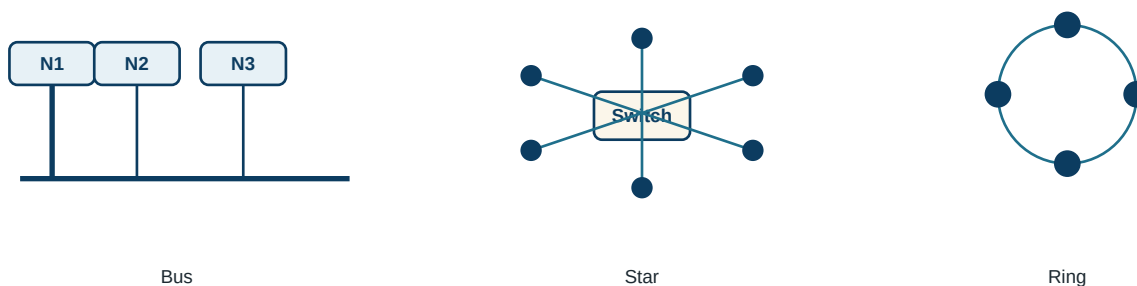


Figure 7.1 Bus, star and ring.

Topologies

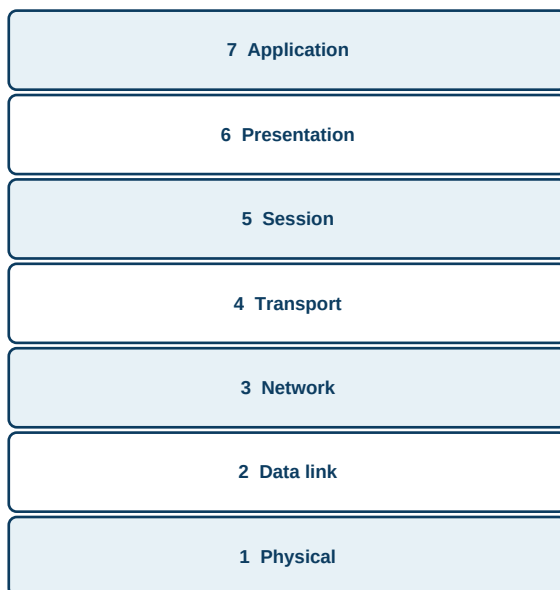
Shape	How it is joined	What to say in an answer
Bus	Every device taps one backbone cable	Easy to add a device. A break in the backbone stops the network.
Star	Every device has its own cable to a central switch	A broken spoke affects one device. The centre is essential.

Shape	How it is joined	What to say in an answer
Ring	Each device connects to the next in a circle	A token or a turn controls who may send. A break can stop the ring.
Mesh	Devices have several paths to each other	The network survives a broken link. It uses more cable.
Tree	Stars joined in a hierarchy	It matches a campus: rooms, floors, then a core switch.
Hybrid	A mixture of the shapes above	A real college network is often a hybrid, not a pure shape.

7.5 Standards and architectures

- **ISO**, the International Organization for Standardization, publishes international standards, including the OSI model.
- **IEEE**, the Institute of Electrical and Electronics Engineers, publishes the 802 standards for local networks. IEEE 802.3 is Ethernet. IEEE 802.11 is wireless LAN.
- **ITU**, the International Telecommunication Union, publishes standards for telecommunications.
- **ASCII** is a character-encoding standard, not an organisation. It assigns numbers to letters, digits and controls. The basic set uses 7 bits and contains 128 characters. It is in this unit because communicating computers must agree on how characters are coded.

OSI model



TCP/IP model

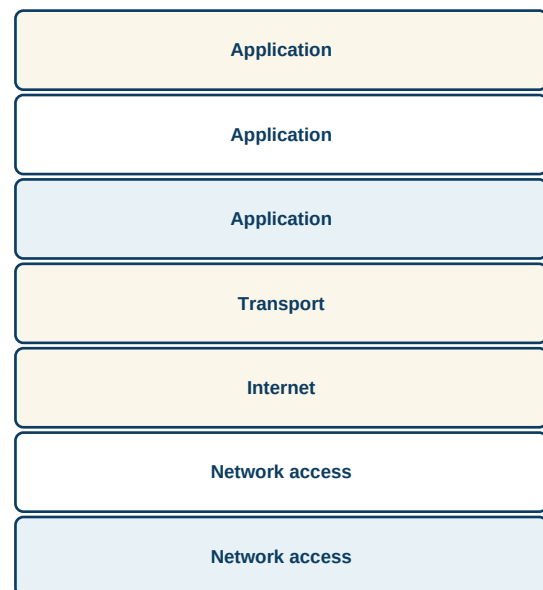


Figure 7.2 OSI beside TCP/IP. Three OSI layers sit inside the TCP/IP application layer.

The OSI layers, from the wire upward

Layer	Job in one sentence
1 Physical	Sends bits over the medium as electrical, light or radio signals.
2 Data link	Organises bits into frames, uses a hardware address, and detects errors on one link.
3 Network	Addresses and routes packets across networks. IP works here.
4 Transport	Carries data end to end between programs. TCP and UDP work here.
5 Session	Opens, manages and closes a dialogue between applications.
6 Presentation	Translates formats, and can compress or encrypt.

Layer	Job in one sentence
7 Application	Provides the service the user sees, such as the web, mail or file transfer.

The **TCP/IP model**, which is the **Internet model** used on the real internet, has four layers. The application layer covers the work of OSI application, presentation and session. The transport layer carries TCP and UDP. The internet layer carries IP and routing. The network-access layer covers the data link and the physical connection. Learn both models. The board may ask for seven OSI names in order, or for the four TCP/IP names.

In the lab

Practical work for this unit is in the journal: straight and crossover cables, the devices in a lab network, an IP address with mask and gateway, and the difference between a network with a central server and a network of equal peers.

Exam point

Memorise the OSI layers with a fixed sentence you can write under pressure, from layer 7 down to layer 1: All People Seem To Need Data Processing. Then still be able to start from the physical layer if the question says 'from the bottom'.

Key terms

Term	Meaning to write in an answer
Protocol	Rules two devices follow in order to communicate.
Simplex	Communication in one direction only.
Full duplex	Communication in both directions at the same time.
LAN	A network covering a small area, owned by one organisation.
WAN	A network covering a large geographical area.
Topology	The layout of nodes and links.
OSI model	A seven-layer reference model for network functions.
TCP/IP	The four-layer model used by the internet.

Bring it together

- Communication needs a sender, a receiver, a medium, a message and a protocol.
- Simplex, half duplex and full duplex describe direction.
- Twisted pair, coax and fibre are guided. Radio, microwave and infrared are unguided.
- LAN and WAN describe the area. Bus, star, ring, mesh, tree and hybrid describe the shape.
- OSI has seven layers. TCP/IP, the Internet model, has four.

Do this in the practical

- 1 Identify straight-through and crossover cables and the devices in the lab: NIC, switch, router, modem.
- 2 Read an IP address, subnet mask and gateway from a working computer.
- 3 Say whether the lab is organised around a central server or as peers.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. A walkie-talkie is an example of:

- A. Simplex
- B. Half duplex
- C. Full duplex
- D. A topology

2. Fibre optic cable carries the signal as:

- A. Pulses of light
- B. Ink
- C. A paper tape
- D. A process state

3. In a star topology, the central device is important because:

- A. A failure there affects the whole star
- B. It stores ROM programs
- C. It replaces the protocol
- D. It is a simplex printer

4. IEEE 802.11 describes:

- A. Wireless LAN
- B. A C++ data type
- C. EPROM erasing
- D. File attributes

5. Which OSI layer routes packets between networks?

- A. Physical
- B. Network
- C. Session
- D. Application

6. TCP and UDP belong to the OSI:

- A. Transport layer
- B. Physical layer
- C. A cable colour
- D. CMOS battery

7. ASCII is:

- A. A character-encoding standard
- B. A topology
- C. A type of RAM
- D. An operating system

8. A LAN is best described as:

- A. A network over a small area under one organisation
- B. A single keyboard
- C. A layer of OSI
- D. A C++ loop

Answer key

Q	Answer	Why this is the answer
1	B	Either person can talk, and only one talks at a time.
2	A	Fibre carries light pulses in a glass or plastic core.
3	A	Every spoke depends on the centre.
4	A	IEEE 802.11 is the wireless LAN family of standards.
5	B	The network layer handles logical addressing and routing.
6	A	Transport provides end-to-end communication. TCP and UDP are its well-known protocols.
7	A	ASCII assigns numeric codes to characters.
8	A	LAN means local area network.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Name the five components of data communication.

Model answer

Sender, receiver, medium, message and protocol. The sender creates the message, the medium carries it, the receiver takes it, and the protocol is the set of rules that makes the exchange understandable.

Q2. Differentiate half duplex and full duplex.

Model answer

Half duplex allows both directions, but only one direction at a time, as in a walkie-talkie. Full duplex allows both directions at the same time, as in a telephone call. Both are two-way. They differ in whether the two directions overlap.

Q3. Differentiate guided and unguided media with one example each.

Model answer

A guided medium contains the signal in a physical path, for example fibre optic cable. An unguided medium carries the signal through the air, for example radio waves used by Wi-Fi. Guided media follow a cable. Unguided media need no cable between the devices.

Q4. Give one advantage and one risk of bus topology.

Model answer

Cable use is simple: devices share one backbone, and a new device can be tapped onto it. A break in that backbone interrupts the network for every device on it.

Q5. Write the four layers of the TCP/IP model.

Model answer

Application, transport, internet and network access. The application layer covers the OSI application, presentation and session work. Transport carries TCP and UDP. The internet layer routes IP packets. Network access puts frames and bits onto the local link.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain types of networks and network topologies with a labelled description of each topology.

- 1 Define a network and distinguish LAN, MAN and WAN by area.
- 2 Define topology.
- 3 Describe bus, star and ring, each with how devices are joined and one strength and one weakness.
- 4 Describe mesh, tree and hybrid in the same way.
- 5 Say which shape matches a school lab and why. A star, or a tree of stars, is the usual answer.

Question 2. Explain the OSI model and compare it with TCP/IP.

- 1 Say that OSI is a reference model of seven layers and name them in order.
- 2 Give one job for each layer, especially physical, data link, network, transport and application.
- 3 Name the four TCP/IP layers and show which OSI layers they cover.
- 4 State that TCP/IP is the model the internet uses. OSI is the teaching and design reference.